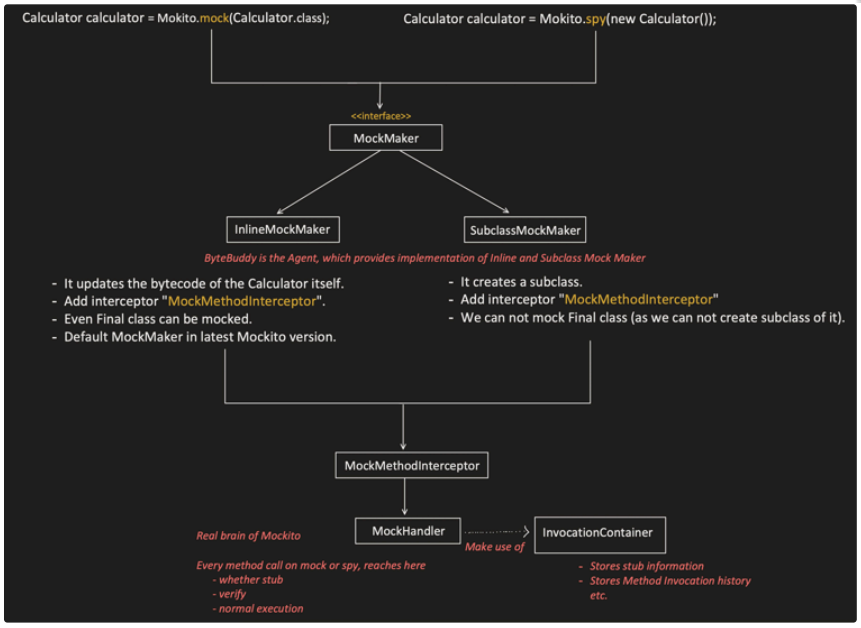
## Mockito Internals

```java
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int multiply(int a, int b) {
        return a * b;
    }
}

```

```java
@Test
void placeOrderSuccess() {

    Calculator calculator = mock(Calculator .class);

    when(calculator.multiply(4, 2)).thenReturn(10);

    int output = calculator.multiply(4, 2);

    assertTrue(result);

    verify(paymentService).pay(10);
 }

```

1. **<u>What happens internally when we create Mock or Spy object</u>**

Concept && Coding




**<u>Lets see dummy code, what ByteBuddy might be doing in case of InlineMockMaker and SubclassMockMaker:</u>**

***Conceptual code what Inline Mock Maker might have done:***

```java
class Calculator  {
```

```java

    MockMethodInterceptor mockitoInterceptor;

    void setMockitoInterceptor(MockMethodInterceptor interceptor) {
              this.mockitoInterceptor = interceptor;
    }


     int multiply(int a, int b) {

           // if interception  toggle is off, run real code, it is required to avoid infinite loop.
         if (interceptFlagDisabled()) {
                   return a * b;
         }

        return mockitoInterceptor.doIntercept(this, multiply, new Object[]{a, b}, realMethod);
     }

     // similar changes for add method

}

```

*Use case:*

*this* **interceptFlagDisabled** *toggle is specifically required for SPY use case, where we call real method, when stubbing is not available.*

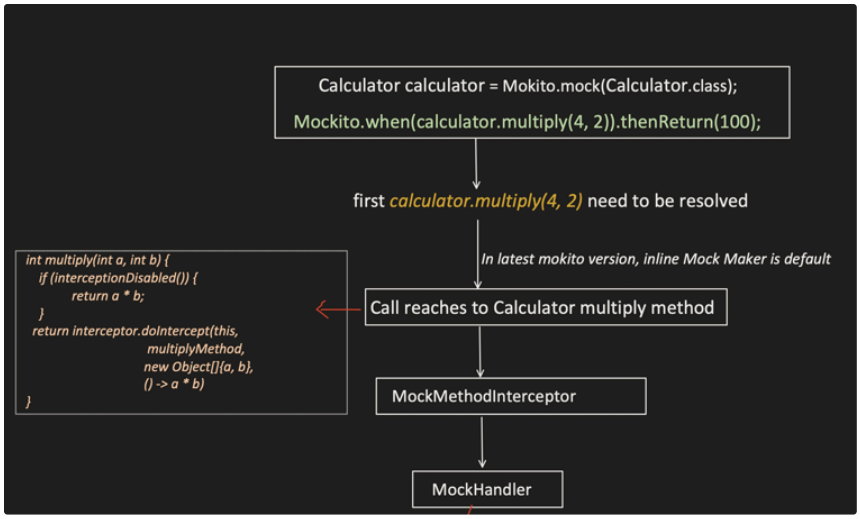
*Unit test → multiply(a, b) → intercept → MockHandler → no stubbing found → Spy object, so default it will invoke real method → Now Interception should be disabled else it will cause infinite loop.*

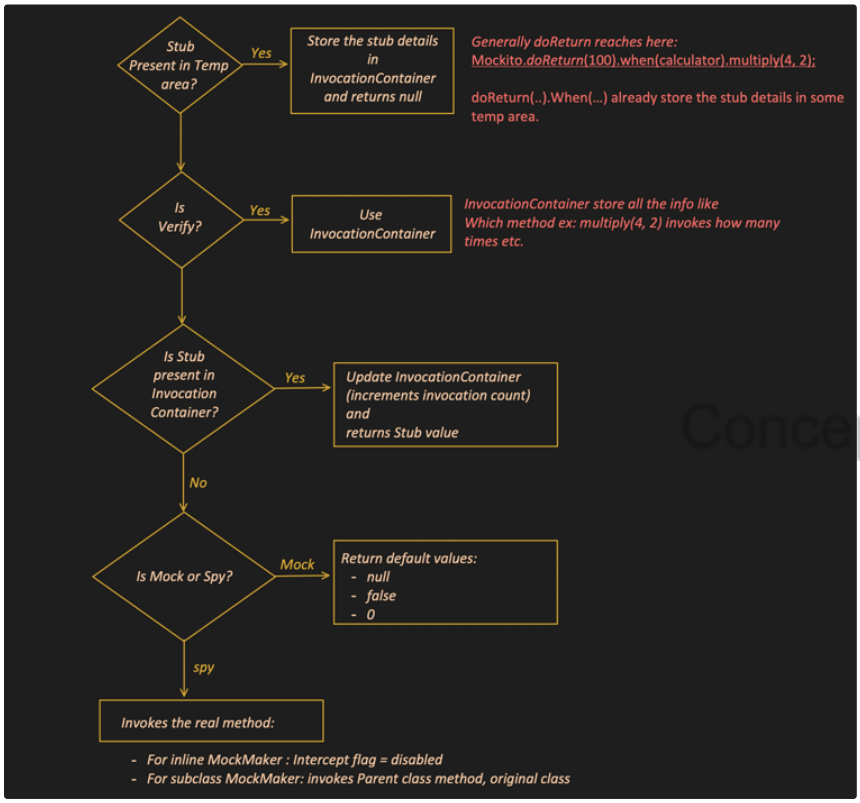
Concept && Coding

***Conceptual code what Subclass Mock Maker might have done:***

```java
class CalculatorSpyProxy extends Calculator  {

    MockMethodInterceptor mockitoInterceptor;

    void setMockitoInterceptor(MockMethodInterceptor interceptor) {
              this.mockitoInterceptor = interceptor;
    }

    @Override
    int add(int a, int b) {
        return mockitoInterceptor.doIntercept(this, add, new Object[]{a, b}, realMethod);
    }

    @Override
    int multiply(int a, int b) {
        return mockitoInterceptor.doIntercept(this, multiply, new Object[]{a, b}, () -> super.multiply(a, b));
    }
}

```

**<u>What happens internally when we do stubbing on Mock or Spy objects</u>**



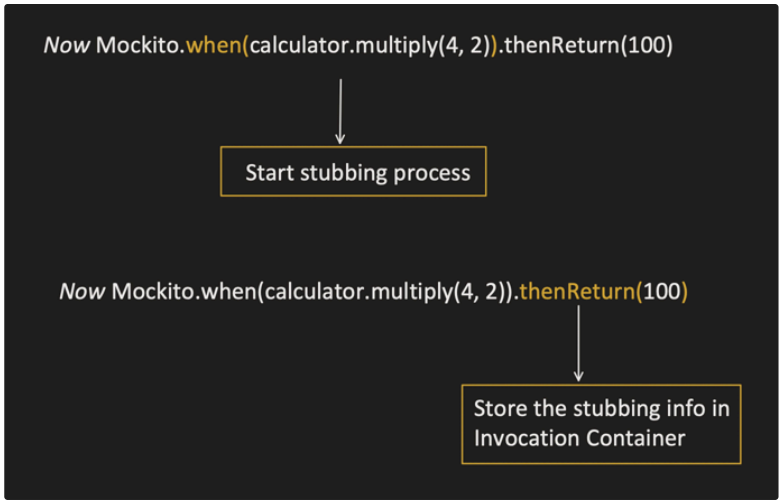

Concept && Coding

**Within Mock Handler the flow is:-**

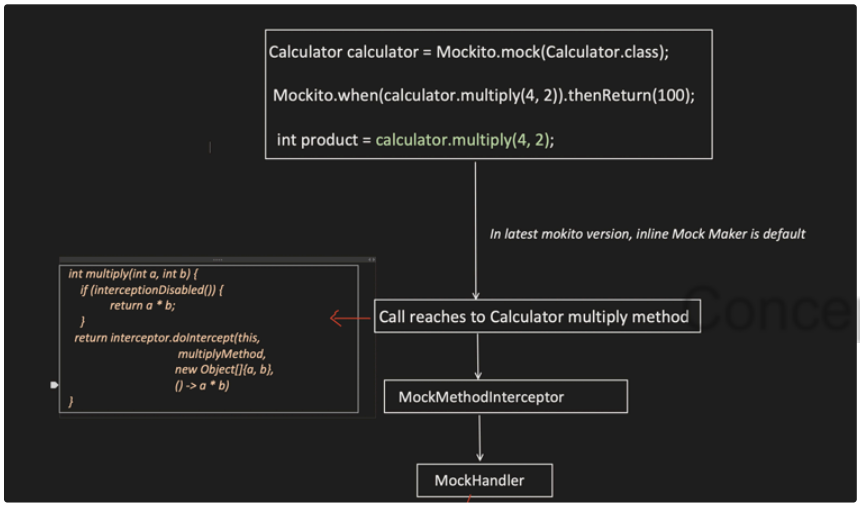
*So for* Mockito.when(calculator.multiply(4, 2)).thenReturn(100)

- It will return 0

Since calculator object is Mock, no spy that's why default value return for int is 0.

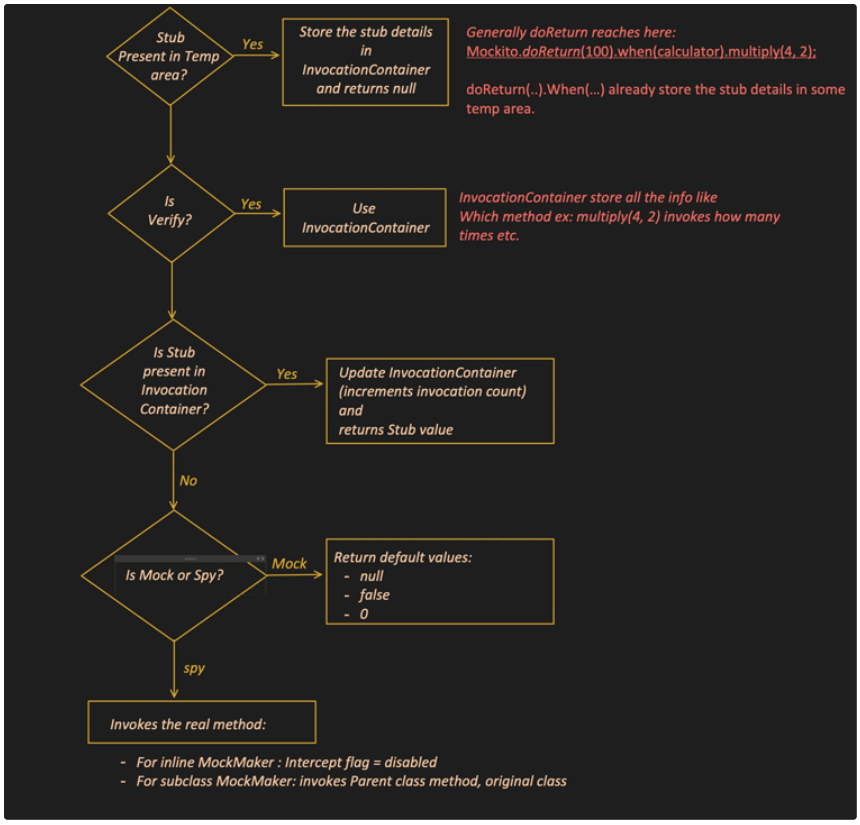



**What happens when method is invoked**




Concept && Coding

**Within Mock Handler the flow is :-**




*So* calculator.multiply(4, 2)

- It will return 100

Concept && Coding